Assignment 10 – High-Availability Docker Swarm on AWS

Tutorial followed: BetterStack – HA Docker Swarm Guide

Overview

This assignment demonstrates setting up a production-style, highly available Docker Swarm cluster on AWS EC2. The cluster consists of **5 nodes** (3 managers + 2 workers), deploys an Nginx service with 4 replicas, and validates high-availability by draining nodes and simulating a node failure.

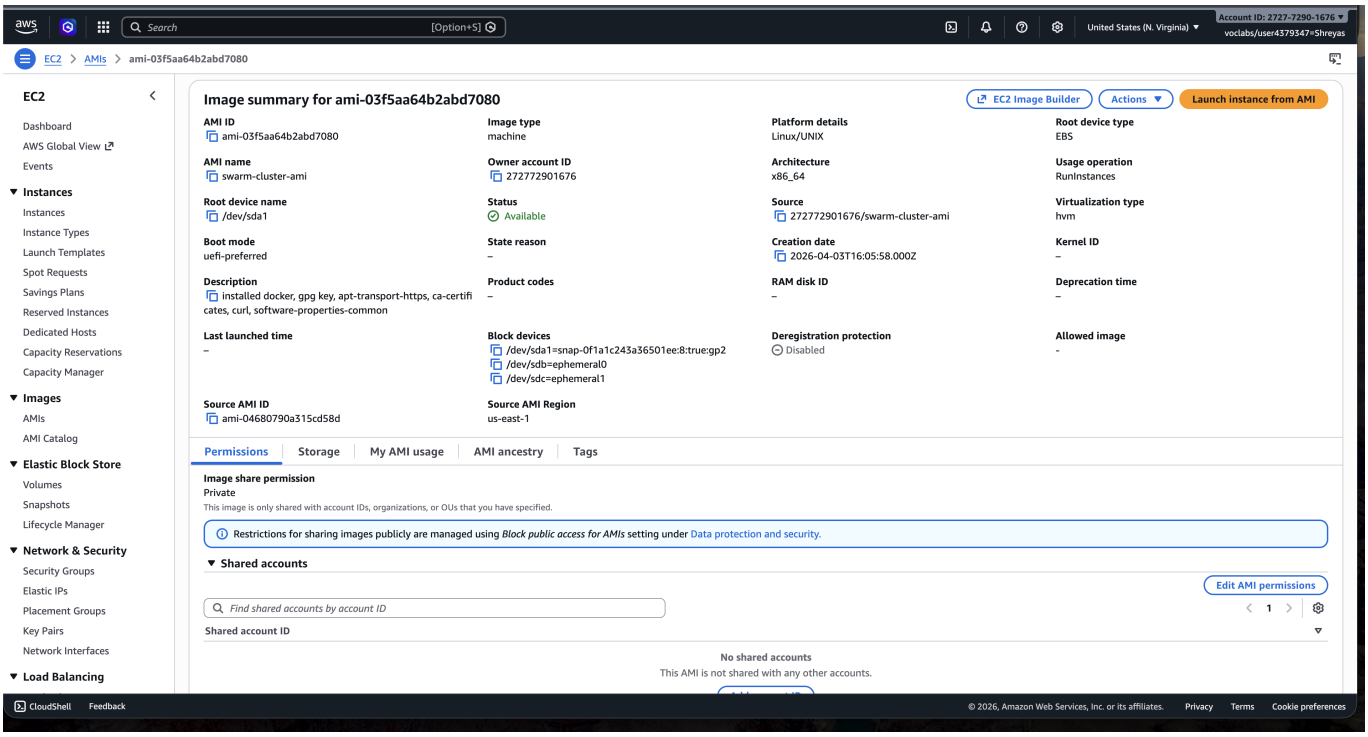
Infrastructure Setup

Custom AMI

A pre-built AMI (`ami-03f5aa64b2abdf080`) was used to launch all EC2 instances. The AMI had the following pre-installed:

- Docker Engine
- Supporting packages: `apt-transport-https`, `ca-certificates`, `curl`, `gnupg`, `software-properties-common`

This ensured all nodes had a consistent Docker environment without manual per-node setup.

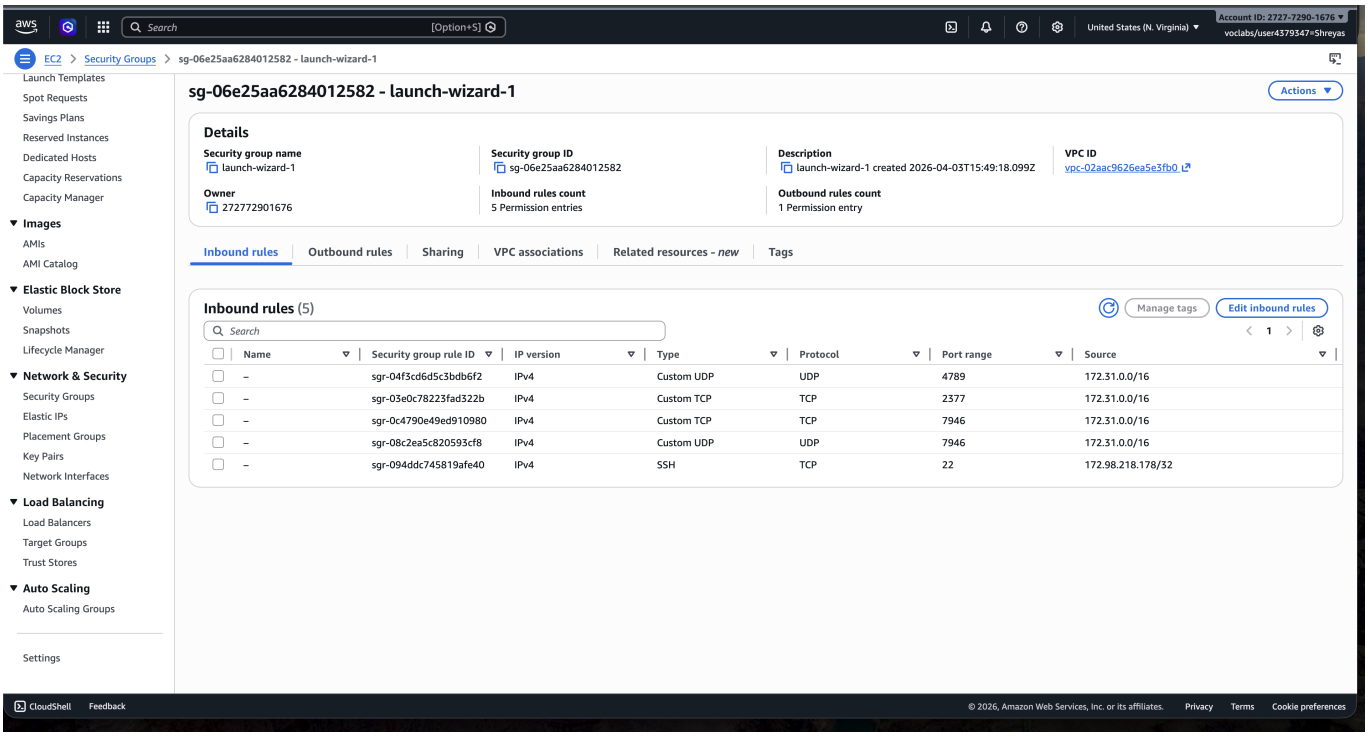


Security Group Configuration

Security group `sg-06e25aa628401582` (`launch-wizard-1`) was configured with the following inbound rules to allow Docker Swarm inter-node communication:

Protocol	Port	Purpose
TCP	22	SSH access
TCP	2377	Swarm cluster management traffic
TCP/UDP	7946	Node-to-node communication
UDP	4389	Overlay network traffic (VXLAN)

All Swarm ports were scoped to the VPC CIDR (`172.31.0.0/16`) for internal-only access.



Cluster Initialization

Step 1 – Verify Docker on Manager Node

SSH into the designated manager node and confirm Docker is active:

```
sudo systemctl status docker
```

Docker was confirmed `active (running)`.

Step 2 – Retrieve Manager's Private IP

```
hostname -I | awk '{print $1}'
# Output: 172.31.71.106
```

Step 3 – Initialize the Swarm

```
docker swarm init --advertise-addr 172.31.71.106
```

Output confirmed:

```
Swarm initialized: current node (2Xyx4emacIpyc1hoovAhbklm) is now a
manager.
```

A join token for workers was printed. To retrieve the manager join token separately:

```
docker swarm join-token manager
```

```
ubuntu@ip-172-31-71-106:~$ sudo systemctl status docker
● docker.service - Docker Application Container Engine
     Loaded: loaded (/lib/systemd/system/docker.service; enabled; vendor preset: enabled)
     Active: active (running) since Fri 2026-04-03 16:07:39 UTC; 2min 14s ago
TriggeredBy: ● docker.socket
       Docs: https://docs.docker.com
   Main PID: 521 (dockerd)
      Tasks: 9
     Memory: 112.6M
        CPU: 574ms
     CGroup: /system.slice/docker.service
             └─521 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

Apr 03 16:07:38 ip-172-31-71-106 dockerd[521]: time="2026-04-03T16:07:38.647348389Z" level=info msg="Restoring containers: start."
Apr 03 16:07:38 ip-172-31-71-106 dockerd[521]: time="2026-04-03T16:07:38.710095067Z" level=info msg="Deleting nftables IPv4 rules" error="exit status 1" outpu>
Apr 03 16:07:38 ip-172-31-71-106 dockerd[521]: time="2026-04-03T16:07:38.727646610Z" level=info msg="Deleting nftables IPv6 rules" error="exit status 1" outpu>
Apr 03 16:07:39 ip-172-31-71-106 dockerd[521]: time="2026-04-03T16:07:39.190459926Z" level=info msg="Loading containers: done."
Apr 03 16:07:39 ip-172-31-71-106 dockerd[521]: time="2026-04-03T16:07:39.210944687Z" level=info msg="Docker daemon" commit=f78c987 containerd-snapshotter=true>
Apr 03 16:07:39 ip-172-31-71-106 dockerd[521]: time="2026-04-03T16:07:39.212064682Z" level=info msg="Initializing buildkit"
Apr 03 16:07:39 ip-172-31-71-106 dockerd[521]: time="2026-04-03T16:07:39.257014960Z" level=info msg="Completed buildkit initialization"
Apr 03 16:07:39 ip-172-31-71-106 dockerd[521]: time="2026-04-03T16:07:39.268240161Z" level=info msg="Daemon has completed initialization"
Apr 03 16:07:39 ip-172-31-71-106 dockerd[521]: time="2026-04-03T16:07:39.268457934Z" level=info msg="API listen on /run/docker.sock"
Apr 03 16:07:39 ip-172-31-71-106 systemd[1]: Started Docker Application Container Engine.
ubuntu@ip-172-31-71-106:~$ hostname -I | awk '{print $1}'
172.31.71.106
ubuntu@ip-172-31-71-106:~$ docker swarm init --advertise-addr 172.31.71.106
Swarm initialized: current node (2xy4wwaclpyclhov6xh8xjqkg) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-3l7dum783pk0f2hquw3vaxnh0hcczhhcda8jlquhp01jejnn43-ae3vjrrm9jct0hvj4ew8p762i 172.31.71.106:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.
ubuntu@ip-172-31-71-106:~$
```

Adding Nodes to the Swarm

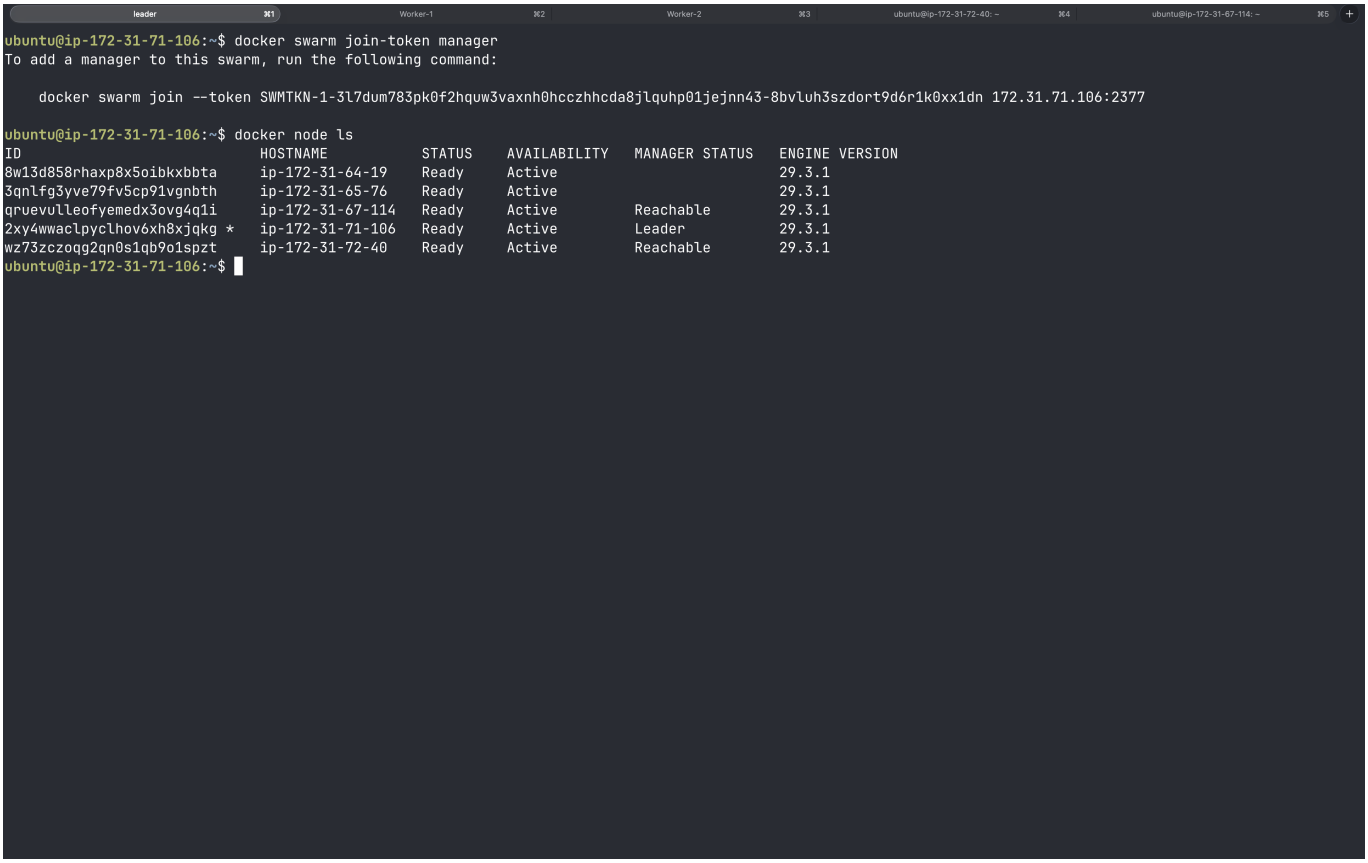
Worker and additional manager nodes were joined using the respective `docker swarm join` commands with the tokens generated above.

Final Cluster – 5 Nodes

```
docker node ls
```

Node IP	Status	Availability	Manager Status	Engine
ip-172-31-64-19	Ready	Active	–	29.3.1
ip-172-31-65-76	Ready	Active	–	29.3.1
ip-172-31-67-114	Ready	Active	Reachable	29.3.1
ip-172-31-71-106	Ready	Active	**Leader**	29.3.1
ip-172-31-72-40	Ready	Active	Reachable	29.3.1

- **3 Manager nodes** (1 Leader + 2 Reachable) — provides quorum fault tolerance
- **2 Worker nodes**



Deploying a Service Stack

nginx.yaml

```yaml
version: '3.9'

networks:
  nginx:
    external: false
```

```yaml
services:
  # --- NGINX ---
  nginx:
    image: nginx:latest
    ports:
      - '8080:80'
    deploy:
      replicas: 4
      update_config:
        parallelism: 2
        order: start-first
        failure_action: rollback
        delay: 10s
      rollback_config:
        parallelism: 0
        order: stop-first
      restart_policy:
        condition: any
        delay: 5s
        max_attempts: 3
        window: 120s
    healthcheck:
      test: ["CMD", "service", "nginx", "status"]
      interval: 10s
```

Deploy the Stack

```
docker stack deploy -c nginx.yaml nginx
```

Verify Service

```
docker service ls
```

```
ID             NAME           MODE         REPLICAS   IMAGE          PORTS
col5pszx8nNe   nginx_nginx    replicated   4/4        nginx:latest   *:8080-
>80/tcp
```

```
docker service ps col5pszx8nNe
```

All 4 replicas were distributed across multiple nodes and reported `Running`.

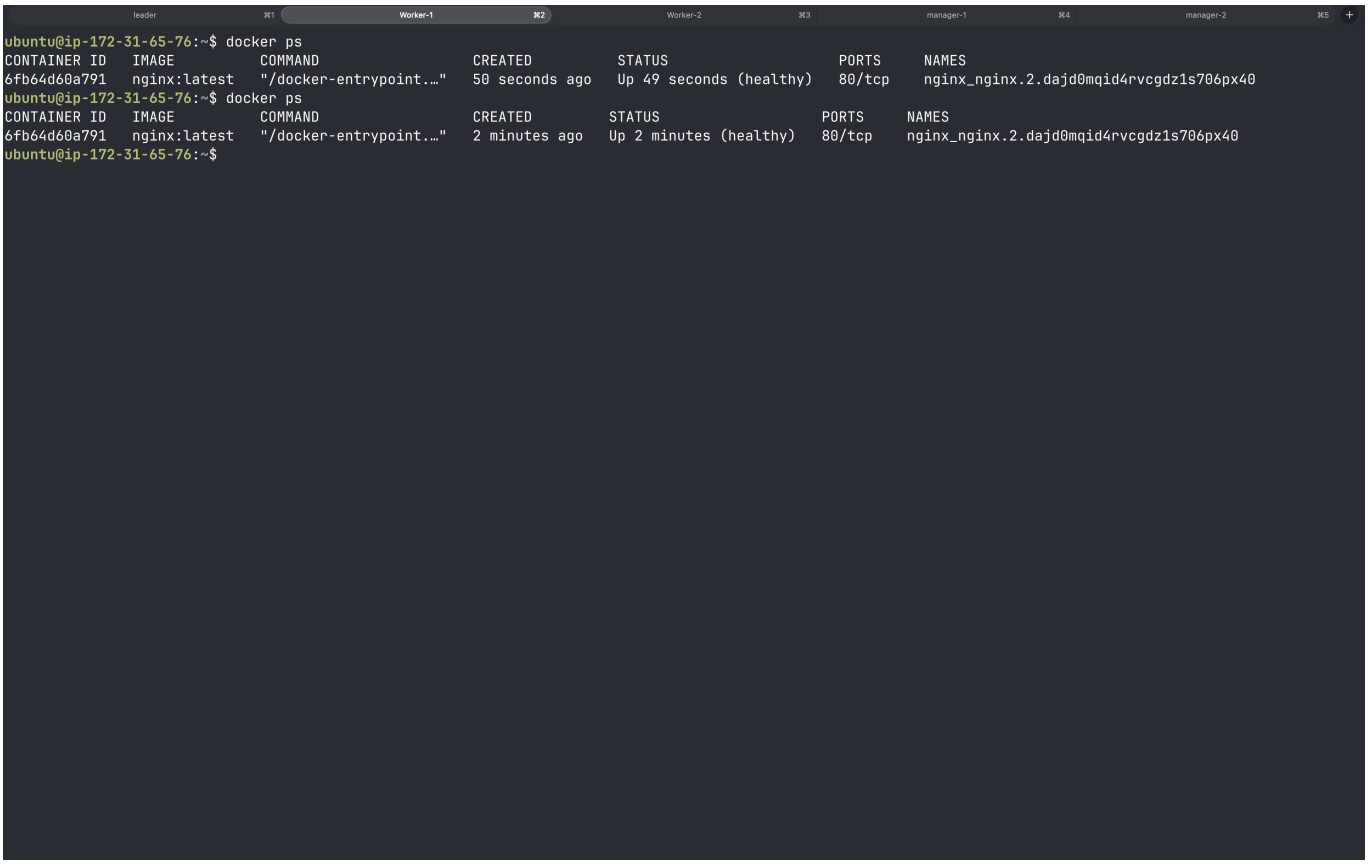
```
leader ⌘1   Worker-1 ⌘2   Worker-2 ⌘3   manager-1 ⌘4   manager-2 ⌘5   +
ubuntu@ip-172-31-72-40:~$ cat nginx.yaml
version: '3.7'

networks:
  nginx:
    external: false

services:

  # --- NGINX ---
  nginx:
      image: nginx:latest
      ports:
        - '8088:80'
      deploy:
        replicas: 4
        update_config:
          parallelism: 2
          order: start-first
          failure_action: rollback
          delay: 10s
        rollback_config:
          parallelism: 0
          order: stop-first
        restart_policy:
          condition: any
          delay: 5s
          max_attempts: 3
          window: 120s
      healthcheck:
        test: ["CMD", "service", "nginx", "status"]
      networks:
        - nginx
ubuntu@ip-172-31-72-40:~$ docker stack deploy -c nginx.yaml nginx
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network nginx_nginx
Creating service nginx_nginx
ubuntu@ip-172-31-72-40:~$ docker service ls
ID             NAME          MODE         REPLICAS   IMAGE          PORTS
co5piszx6n9e   nginx_nginx   replicated   0/4        nginx:latest   *:8088->80/tcp
ubuntu@ip-172-31-72-40:~$ docker service ls
ID             NAME          MODE         REPLICAS   IMAGE          PORTS
co5piszx6n9e   nginx_nginx   replicated   0/4        nginx:latest   *:8088->80/tcp
ubuntu@ip-172-31-72-40:~$ docker service ps co5piszx6n9e
ID             NAME            IMAGE          NODE               DESIRED STATE   CURRENT STATE               ERROR     PORTS
iymhwwg8ihs7   nginx_nginx.1   nginx:latest   ip-172-31-72-40    Running         Running about a minute ago
dajd0mqid4rv   nginx_nginx.2   nginx:latest   ip-172-31-65-76    Running         Running about a minute ago
or77s646v7px   nginx_nginx.3   nginx:latest   ip-172-31-64-19    Running         Running about a minute ago
vb8iokk3imz9   nginx_nginx.4   nginx:latest   ip-172-31-67-114   Running         Running about a minute ago
ubuntu@ip-172-31-72-40:~$
```

Verify Containers on a Worker Node

SSH into `ip-172-31-65-76` (a worker node) and run:

```
docker ps
```

Two nginx containers were confirmed running and `healthy` on that node.



High-Availability Testing

Test 1 – Node Draining

Nodes were drained to simulate maintenance (e.g., patching, upgrades). Swarm automatically rescheduled the tasks onto remaining active nodes.

```
docker node update --availability drain gruevulleofyemedx3ovg4q11
docker node update --availability drain wz73zccogp2qnd1q0h9ol1pzt
```

After draining, `docker service ls` still showed `4/4` replicas — Swarm migrated all tasks to the remaining active nodes with zero downtime.

```
docker service ps col5pszx8nNe
```

Old tasks on drained nodes transitioned to `Shutdown`; replacement tasks on active nodes were `Running`.

```
leader ⌘1 Worker-1 ⌘2 Worker-2 ⌘3 manager-1 ⌘4 manager-2 ⌘5 +

ubuntu@ip-172-31-72-40:~$ docker service ls
ID             NAME          MODE         REPLICAS   IMAGE          PORTS
co5piszx6n9e   nginx_nginx   replicated   4/4        nginx:latest   *:8088->80/tcp
ubuntu@ip-172-31-72-40:~$ docker node ls
ID                            HOSTNAME           STATUS    AVAILABILITY   MANAGER STATUS   ENGINE VERSION
8w13d858rhaxp8x5oibkxbbta     ip-172-31-64-19    Ready     Active                          29.3.1
3qnlfg3yve79fv5cp91vgnbth     ip-172-31-65-76    Ready     Active                          29.3.1
qruevulleofyemedx3ovg4q1i     ip-172-31-67-114   Ready     Active         Reachable        29.3.1
2xy4wwaclpyclhov6xh8xjqkg     ip-172-31-71-106   Ready     Drain          Leader           29.3.1
wz73zczoqg2qn0s1qb9o1spzt *   ip-172-31-72-40    Ready     Active         Reachable        29.3.1
ubuntu@ip-172-31-72-40:~$ docker node update --availability drain qruevulleofyemedx3ovg4q1i
qruevulleofyemedx3ovg4q1i
ubuntu@ip-172-31-72-40:~$ docker node update --availability drain wz73zczoqg2qn0s1qb9o1spzt
wz73zczoqg2qn0s1qb9o1spzt
ubuntu@ip-172-31-72-40:~$ docker node ls
ID                            HOSTNAME           STATUS    AVAILABILITY   MANAGER STATUS   ENGINE VERSION
8w13d858rhaxp8x5oibkxbbta     ip-172-31-64-19    Ready     Active                          29.3.1
3qnlfg3yve79fv5cp91vgnbth     ip-172-31-65-76    Ready     Active                          29.3.1
qruevulleofyemedx3ovg4q1i     ip-172-31-67-114   Ready     Drain          Reachable        29.3.1
2xy4wwaclpyclhov6xh8xjqkg     ip-172-31-71-106   Ready     Drain          Leader           29.3.1
wz73zczoqg2qn0s1qb9o1spzt *   ip-172-31-72-40    Ready     Drain          Reachable        29.3.1
ubuntu@ip-172-31-72-40:~$ docker service ls
ID             NAME          MODE         REPLICAS   IMAGE          PORTS
co5piszx6n9e   nginx_nginx   replicated   2/4        nginx:latest   *:8088->80/tcp
ubuntu@ip-172-31-72-40:~$ docker service ps co5piszx6n9e
ID             NAME                IMAGE          NODE               DESIRED STATE   CURRENT STATE            ERROR     PORTS
1u9na29si34b   nginx_nginx.1       nginx:latest   ip-172-31-65-76    Running         Running 2 seconds ago
iymhwwg8ihs7    \_ nginx_nginx.1   nginx:latest   ip-172-31-72-40    Shutdown        Shutdown 33 seconds ago
dajd0mqid4rv   nginx_nginx.2       nginx:latest   ip-172-31-65-76    Running         Running 31 minutes ago
or77s646v7px   nginx_nginx.3       nginx:latest   ip-172-31-64-19    Running         Running 31 minutes ago
bjccfq0wywge   nginx_nginx.4       nginx:latest   ip-172-31-64-19    Running         Running 2 seconds ago
v95ry1c2gbed    \_ nginx_nginx.4   nginx:latest   ip-172-31-72-40    Shutdown        Shutdown 33 seconds ago
vb8iokk3imz9    \_ nginx_nginx.4   nginx:latest   ip-172-31-67-114   Shutdown        Shutdown 41 seconds ago
ubuntu@ip-172-31-72-40:~$ docker service ls
ID             NAME          MODE         REPLICAS   IMAGE          PORTS
co5piszx6n9e   nginx_nginx   replicated   4/4        nginx:latest   *:8088->80/tcp
ubuntu@ip-172-31-72-40:~$
```

```
leader ⌘1 Worker-1 ⌘2 Worker-2 ⌘3 manager-1 ⌘4 manager-2 ⌘5 +

ubuntu@ip-172-31-72-40:~$ docker node ls
ID                            HOSTNAME           STATUS    AVAILABILITY   MANAGER STATUS   ENGINE VERSION
8w13d858rhaxp8x5oibkxbbta     ip-172-31-64-19    Ready     Active                          29.3.1
3qnlfg3yve79fv5cp91vgnbth     ip-172-31-65-76    Ready     Active                          29.3.1
qruevulleofyemedx3ovg4q1i     ip-172-31-67-114   Ready     Drain          Reachable        29.3.1
2xy4wwaclpyclhov6xh8xjqkg     ip-172-31-71-106   Ready     Drain          Reachable        29.3.1
wz73zczoqg2qn0s1qb9o1spzt *   ip-172-31-72-40    Ready     Drain          Leader           29.3.1
ubuntu@ip-172-31-72-40:~$ docker service ls
ID             NAME          MODE         REPLICAS   IMAGE          PORTS
co5piszx6n9e   nginx_nginx   replicated   4/4        nginx:latest   *:8088->80/tcp
ubuntu@ip-172-31-72-40:~$ docker service ps co5piszx6n9e
ID             NAME                IMAGE          NODE               DESIRED STATE   CURRENT STATE           ERROR     PORTS
1u9na29si34b   nginx_nginx.1       nginx:latest   ip-172-31-65-76    Running         Running 2 minutes ago
iymhwwg8ihs7    \_ nginx_nginx.1   nginx:latest   ip-172-31-72-40    Shutdown        Shutdown 6 minutes ago
dajd0mqid4rv   nginx_nginx.2       nginx:latest   ip-172-31-65-76    Running         Running 2 minutes ago
or77s646v7px   nginx_nginx.3       nginx:latest   ip-172-31-64-19    Running         Running 2 minutes ago
bjccfq0wywge   nginx_nginx.4       nginx:latest   ip-172-31-64-19    Running         Running 2 minutes ago
v95ry1c2gbed    \_ nginx_nginx.4   nginx:latest   ip-172-31-72-40    Shutdown        Shutdown 6 minutes ago
vb8iokk3imz9    \_ nginx_nginx.4   nginx:latest   ip-172-31-67-114   Shutdown        Shutdown 6 minutes ago
ubuntu@ip-172-31-72-40:~$
```

Test 2 – Simulating a Node Failure

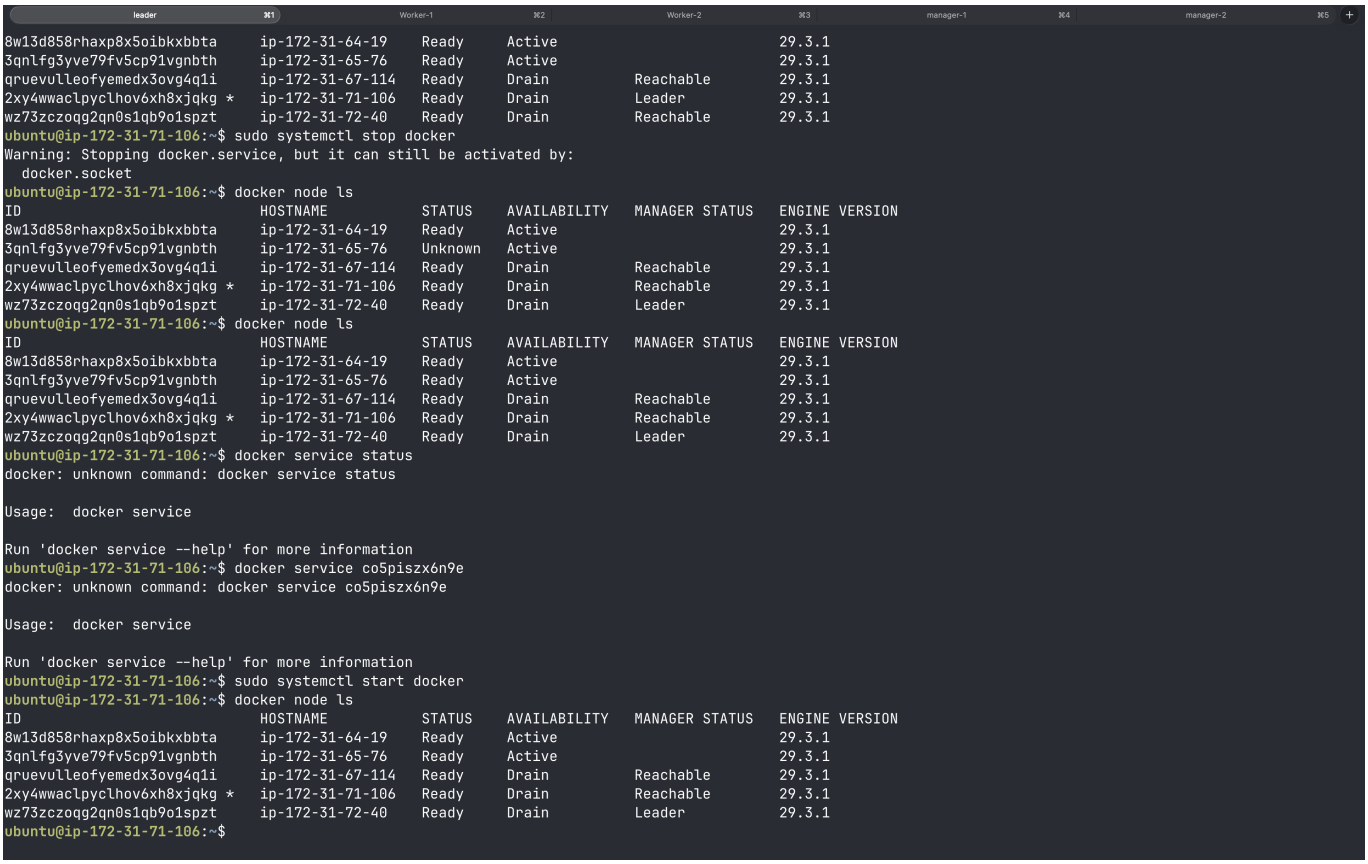
Docker was stopped on one of the manager nodes to simulate an unexpected failure:

```
sudo systemctl stop docker
```

Observations from the remaining manager:

- The stopped node's status changed to `Unknown` in `docker node ls`.
- The Swarm cluster remained operational (quorum was maintained with 2 of 3 managers still up).
- After restarting Docker on the node (`sudo systemctl start docker`), it rejoined the cluster and returned to `Ready` state.

```
sudo systemctl start docker
docker node ls   # Node back to Ready
```



Environment Summary

Component	Value
Cloud Provider	AWS EC2
AMI	`ami-03f5aa64b2abdf080` (custom, Docker pre-installed)
Number of Nodes	5 (3 managers, 2 workers)
Docker Engine Version	29.3.1
Swarm Manager (Leader)	

Component	Value
Service Deployed	Nginx (4 replicas, port 8080)
Security Group	